Contents

Failed Approaches at N≥256: A Detailed Account

Document Purpose

This document is for an advisor meeting. It explains every approach we tried to scale our neural MAC polar decoder beyond N=128 — what we tried, why we tried it, what happened, and why it failed. Each section is written so you can answer follow-up questions from your advisor.

---

1. Background and Problem Statement

What works

We have a neural successive cancellation (SC) decoder for two-user MAC polar codes. The decoder replaces the analytical CalcLeft, CalcRight, and CalcParent operations on 2x2 probability tensors with learned MLPs operating on d-dimensional embeddings.

N	NN-SC BLER	SC BLER	Ratio
16	0.0114	0.0106	1.08x
32	0.0088	0.008	1.10x
64	0.026	0.025	1.03x
128	0.017	0.016	1.04x

The neural decoder matches analytical SC within 4 percent at all block lengths up to N=128 on the GMAC at SNR=6dB, Class B (interleaved path with R_u ≈ R_v ≈ 0.48). On BEMAC the neural decoder actually beats SC for N≥64.

What does not work

At N=256, the d=16 model converges to BLER ≈ 0.015 with curriculum training, which is 3x worse than analytical SC (0.005). At N=512 the gap grows to 8x. At N=1024 we cannot train at all without curriculum.

The question we have been investigating for the past two weeks: **why does the neural decoder fail at N≥256 and can we fix it?**

---

2. The Core Difficulty

The neural SC tree decoder operates sequentially. Each leaf decision depends on all previous leaf decisions through the CalcParent operation. For block length N there are 2N leaf decisions, each requiring O(log N) tree operations, giving a total of O(N log N) sequential MLP calls.

For N=256 this is roughly 1500 sequential MLP calls. The gradient must flow through all of them during training. This creates several problems at once:

1. **Vanishing gradients** through 1500 sequential operations
2. **Long training times** (one iteration takes seconds)
3. **Error accumulation** during inference (each leaf decision can corrupt downstream decisions)

For comparison, the single-user Neural Polar Decoder (NPD) of Aharoni et al. 2024 uses a clever training trick called fast_ce that gives O(log N) gradient depth instead of O(N log N). For N=1024 this is 10 sequential steps instead of 10,240. The single-user NPD scales to N=1024 effortlessly.

We spent two weeks trying to apply fast_ce, or any equivalent O(log N) training method, to our 4-class MAC decoder. Everything failed.

---

3. Approach 1 — Direct Fast_CE for 4-Class MAC

What it is

The single-user NPD's fast_ce trick exploits the fact that during teacher-forced training, all true bits are known. So at each tree depth d, you can compute CheckNode and BitNode for all positions at that depth in parallel. This gives log_2(N) sequential steps instead of 2N-1.

We tried to apply the same technique to our 4-class MAC decoder, where each leaf produces a joint (u,v) decision (4 classes instead of 2).

What we did

Implemented fast_ce with our existing tree walk architecture, modified to handle 4-class targets at every position. Trained at N=32 for 30K iterations.

What happened

Loss converged to ~0.30 (vs random ~1.4). BLER was 0.34 — about 7x worse than SC (0.046). The gap is large and stable.

Why it failed

We did detailed diagnostics later. The root cause is that 4-class decisions create a fundamentally different error structure than binary decisions:

- **Single-user (2-class):** A wrong bit just flips a sign in the BitNode residual `e1*sign(u) + e2`. The embedding stays well-formed — same magnitude, just mirrored. The model trained with teacher forcing still works during free-running because wrong predictions produce inputs the model has seen.
- **4-class MAC:** A wrong (u,v) prediction creates one of three distinct error patterns across the four Z_2 x Z_2 characters. Each pattern produces a qualitatively different embedding perturbation. The model trained with teacher forcing has never seen these mixed patterns and cannot recover from them.

This is the fundamental obstacle: fast_ce relies on the assumption that "small errors during training stay small during inference." For binary outputs this holds. For 4-class outputs it does not.

Likely advisor questions

Q: Why don't you just train longer? A: Because the model converges. Loss stops decreasing and BLER stabilizes at the 7x ceiling regardless of training length. We tried up to 50K iterations and the curve is flat after iteration 10K.

Q: Did you try a larger model? A: Yes — d=32, hidden=128, 153K parameters. Same 7x ceiling. The bottleneck is not model capacity.

Q: Is the loss landscape pathological? A: We tracked per-depth accuracy. At every N tested, the shallowest depth has ~73 percent accuracy and deeper depths have 95-99 percent. The 73 percent at the shallow depth is the same regardless of model size. It is an information-theoretic plateau caused by the joint 4-class decision structure.

---

4. Approach 2 — Walsh-Hadamard Transform Decomposition

What it is

The CalcLeft operation in our MAC decoder is a circular convolution over the Klein four-group Z_2 x Z_2. By a standard result, this convolution becomes element-wise multiplication in the Walsh-Hadamard transform domain. Element-wise operations are trivially parallelizable.

We thought: if we represent embeddings in the WHT domain, then CalcLeft becomes per-channel element-wise multiplication. The 4-class joint output decomposes into 4 independent scalar channels (one per WHT coefficient). Each channel can be trained with binary fast_ce, which we know works.

What we did

Implemented a WHT-domain decoder where: - The d-dimensional embedding is divided into 4 groups of d/4 dimensions, one per WHT coefficient. - CalcLeft operates element-wise across the 4 groups. - BitNode uses character-dependent sign flips: each WHT channel has its own sign pattern based on the joint (u,v) decision. - Tested at N=32 with several variants: shared MLPs, per-channel MLPs, large model with 283K parameters.

What happened

Variant	Parameters	BLER	vs SC
Shared, binary flip	6K	0.606	13.2x
Shared, character flip	23K	0.336	7.3x
Per-channel, large	283K	0.254	5.5x

Best result: BLER = 0.254 (5.5x SC). Better than direct fast_ce (7.4x) but still nowhere near SC.

Why it failed

The WHT decomposition is mathematically correct. CalcLeft really does become element-wise in the WHT domain. But this only solves the CalcLeft coupling. The BitNode still produces 3 distinct error patterns when the 4-class decision is wrong. The fundamental train-test mismatch persists.

We confirmed this by also implementing direct fast_ce on the original architecture (no WHT). Both give the same ~7x ceiling, proving the WHT decomposition was not the source of improvement we needed.

Likely advisor questions

Q: Did you handle the log-domain issue? A: Yes. The WHT diagonalization works in probability domain, but neural decoders work in log domain for stability. In log domain the WHT involves logsumexp which is not separable. We worked around this by storing (sign, log-magnitude) pairs per WHT coefficient, allowing pure additive operations in log space. The architecture was correct.

Q: Why does the per-channel BitNode not help? A: Because the BitNode's job is to condition on the previous decision. Even with separate MLPs per WHT channel, when the joint (u,v) decision is wrong, the wrong sign pattern is fed to all four channels simultaneously. There is no way to recover the correct decision after that.

Q: Is the d=4 (per channel) too small? A: No. We tested d=16 per channel with 283K parameters total. Same ceiling. Capacity is not the issue.

---

5. Approach 3 — Two-Phase Iterative Refinement

What it is

Decompose the joint MAC decoder into two single-user decoders that iterate: 1. Decode User U from the marginal channel z (treating User V as noise). 2. Decode User V from the conditional channel (z, X_hat) using the U estimate. 3. Optionally re-decode U conditioned on V_hat, then re-decode V, etc.

Each phase is a standard single-user decoder, so each phase can use fast_ce with O(log N) training depth. The hope was that 2-3 iterations of this scheme would converge to a joint decoding equivalent to the original MAC decoder.

What we did

Implemented three NPD-style decoders (U marginal, V conditional, U refinement) and trained them jointly with teacher forcing. Tested at N=32.

What happened

Refinement iterations	BLER	vs SC
0 (U alone, no V help)	0.948	20.6x
1 (one U→V→U cycle)	0.656	14.3x
2 (two cycles)	0.518	11.3x

Iteration helps clearly (0.95 → 0.66 → 0.52) but the absolute BLER is much worse than even direct fast_ce. Far from SC (0.046).

Why it failed

Phase 1 (decode U from the marginal channel alone) is inherently broken for Class B. The Class B operating point has R_u = 0.469, but the marginal channel capacity I(Z;X) = 0.464. **R_u is above the marginal channel capacity.** No decoder, neural or analytical, can decode U from the marginal channel alone at this rate.

This means Phase 1 always starts at ~95 percent BLER. Phase 2 then has to decode V given a corrupted U estimate. The errors are too large for iterative refinement to recover.

We could have used a lower rate to make Phase 1 work, but then we would not be solving the original Class B problem at the symmetric rate point. The whole motivation for Class B is the symmetric rate, which is precisely where this decomposition breaks.

Likely advisor questions

Q: Why not use a smarter first phase? A: For Class B at the symmetric rate, the marginal capacity is below the operating rate. Any "first phase" that decodes U alone is information-theoretically impossible. We could use a different code class but then we are not solving the original problem.

Q: Could iterative refinement still work eventually? A: We tested up to 2 iterations explicitly. The improvement per iteration is shrinking (0.95 → 0.66 saves 0.29; 0.66 → 0.52 saves 0.14). Extrapolating, even infinite iterations would not reach the ~0.05 region. The iteration is converging to the wrong fixed point because the initial error is too large.

---

6. Approach 4 — Hybrid: Fast_CE Pretrain → Sequential Fine-Tune

What it is

Maybe fast_ce alone is not enough, but it could provide a good initialization for the sequential decoder. The idea: train a fast_ce model first (cheap, O(log N) gradients), transfer its z_encoder weights to the standard sequential decoder, then fine-tune sequentially.

What we did

Three controlled experiments at N=32: 1. **Sequential from scratch** (baseline) — random init, sequential training for 15K iters 2. **Hybrid with z_encoder transfer** — fast_ce 10K iters, transfer z_encoder, sequential 15K iters 3. **Hybrid with z_encoder + emb2logits transfer** — same but transfer two modules

Then we ran the same hybrid approach at N=256 with a 30-hour budget.

What happened at N=32

Approach	Best BLER	Convergence at 3K	Convergence at 10K
From scratch	0.080	1.000	0.088
z_encoder transfer	**0.062**	0.724	0.062
z_enc + emb2logits	0.074	0.980	0.086

The z_encoder transfer gave a real ~22 percent improvement (0.062 vs 0.080) and faster phase transition (3K vs 5K iters). Adding emb2logits transfer hurt slightly (the fast_ce-trained emb2logits expects different embedding statistics than the sequential one).

What happened at N=256

We ran a 30-hour stress test with 4 variants (different LR and batch sizes). **BLER = 1.0 in every single evaluation across all variants**. The model could not learn at all. Loss was stuck at ~0.874 throughout (vs random 1.04, vs successful training ~0.02).

Why it failed at N=256

The fast_ce z_encoder transfer helps slightly at small N but is useless at large N. The bottleneck at N=256 is not the z_encoder — it is the tree operations.

We discovered this through a diagnostic experiment:

Random model at N=256, evaluated under teacher forcing (with true past bits given): 37 percent accuracy. This is barely above the 25 percent random baseline for 4-class output.

The interpretation: Even when given perfect side information, the random tree ops at N=256 cannot correctly process the channel embedding. The 8-level deep stack of random MLPs destroys the input signal before it reaches the leaf. The model cannot escape this dead start.

Curriculum training works because by the time you reach N=256, the tree ops are already trained at N=128 — they preserve information through 8 levels because they have learned to.

Likely advisor questions

Q: Why does the small improvement at N=32 not transfer to N=256? A: At N=32 the tree depth is 5. The information signal can reach the leaf even with random MLPs because there are only 5 transformations. The z_encoder gives a small head start. At N=256 the depth is 8 — three more layers of random transformations completely destroy the signal. No z_encoder initialization can fix that.

Q: Why not transfer the tree ops too? A: We tried this in our final experiment (Approach 7 below). Tree op transfer combined with retraining gives BLER ≈ 0.044, still worse than the standard curriculum baseline (0.015).

---

7. Approach 5 — Larger Model (d=32)

What it is

Increase model capacity from d=16 (39K parameters) to d=32 (153K parameters). This is the obvious thing to try when your model saturates.

What we did

Trained the d=32 model with curriculum (N=32 → 64 → 128) over ~30 hours of compute.

What happened

N	d=16 BLER	d=32 BLER	SC BLER	d=32 vs SC
32	0.046	**0.037**	0.046	**0.80x** (beats SC)
64	0.026	**0.020**	0.025	**0.80x** (beats SC)
128	0.017	0.0185	0.016	1.16x (still training)

The d=32 model **beats SC by 20 percent** at N=32 and N=64. This is a real result — larger capacity does help. But the training time is much longer (28 hours to reach N=128, vs hours for d=16).

The d=32 training never finished at N=256. The process died from system instability before completing the curriculum to N=256. The trajectory at N=128 was still improving (0.033 → 0.022 → 0.019 → 0.0185 over 50K iters), so we never got to test if it would beat SC at N=128 or run on N=256.

Why this is not a complete answer

- It does not provide a fundamentally better training method.
- Training time grows linearly with N. Reaching N=1024 with d=32 would take weeks of continuous compute.
- It does not address the underlying problem: we have no efficient training recipe.

Likely advisor questions

Q: How confident are you that d=32 would match SC at N=256? A: Moderately confident based on the trajectory at N=128. The model was still improving at iter 50K of 111K. Whether it would have matched SC at N=128 (0.016) is uncertain — it was at 0.019, slowly improving. Whether it would also beat SC at N=256 is even less certain.

Q: Why didn't you finish the d=32 training? A: The process kept dying from system issues over multi-day runs. We could restart it now if you want. Estimated 2-3 more days to complete N=128 + N=256 stages.

---

8. Approach 6 — Gradient Detaching

What it is

Sequential training with full gradients is too expensive at N=256. What if we detach gradients every K steps in the tree walk? This limits maximum gradient depth to K instead of O(N log N), while still using sequential rollouts.

What we did

Tested K=4, 8, 16, 32 at N=32 with 10K iterations each. Compared against full gradients.

What happened

K	BLER (10K iters)
Full	**0.112**
K=4	1.000
K=8	1.000
K=16	0.992
K=32	1.000

With full gradients, the model learns (BLER drops from 1.0 to 0.11 by 10K iters). With ANY gradient detaching, it never crosses the phase transition — BLER stays at 1.0 even after 30K iters.

Why it failed

The phase transition from "stuck at 1.0" to "actually learning" requires a coordinated update across all tree levels simultaneously. Detaching gradients every K steps prevents the upper levels of the tree from receiving any signal about how their outputs will be used by lower levels. The model cannot learn the joint structure.

Likely advisor questions

Q: What about backprop through time variants like TBPTT? A: Mathematically equivalent to gradient detaching for our purposes. We tested the strict version. We did not test "soft" approaches like reducing gradient strength but they are unlikely to help — the issue is qualitative (phase transition) not quantitative.

---

9. Approach 7 — Tree-Op Transfer (Validated Today)

What it is

The diagnostic that revealed the tree ops are the bottleneck suggested this experiment: load the trained tree operations from N=128, replace the z_encoder with a fresh one, train at N=256.

If the tree ops are the only thing that needs curriculum and they transfer cleanly across N, this should converge quickly.

What we did

Loaded the 41 learned parameters from `ncg_gmac_mlp_N128.pt`, re-initialized the z_encoder, scaled it to match expected norms, trained for 5000 iterations at N=256, evaluated with 5000 codewords.

What happened

Training trajectory (200-codeword evals): | Iter | BLER | |——|——| | 500 | 0.120 | | 1500 | 0.055 | | 2000 | 0.030 | | 3000 | 0.030 | | 5000 | 0.030 |

Final evaluation (5000 codewords): - **BLER = 0.0438** - vs SC: 0.005 → 8.8x worse - vs curriculum d=16 baseline: 0.015 → 2.9x worse

Why it failed

The 200-codeword intermediate evaluations were too noisy. They suggested BLER ≈ 0.030 (which would be only 2x worse than the curriculum baseline), but the proper 5000-codeword evaluation revealed BLER = 0.044. The previous "BLER = 0.000 at iter 1500" report from a similar earlier experiment was based on only 64 codewords — pure noise.

The actual learning is significant (from random init we would get 1.0; we got 0.044) but it does not match what we get from full curriculum training. The tree ops from N=128 are a useful starting point, but 5000 iterations is not enough to fully adapt them to N=256.

What this teaches us

Tree-op transfer works as a warm start, but it does not eliminate the need for substantial training at the new N. The full curriculum (which trains at N=256 for 100K iterations) reaches BLER = 0.015, while tree-op transfer with 5K iterations reaches 0.044.

Likely advisor questions

Q: What if you train tree-op transfer for 100K iterations? A: That would be the natural follow-up. It might match the full curriculum (0.015), but then we have not saved any time — we have just changed the initialization. The original promise of "curriculum-free fast training" would not hold.

Q: Is there any value in this result? A: It confirms that tree ops are the bottleneck (not z_encoder), which is the diagnostic finding. As a training shortcut it is not useful — full curriculum is faster and gives better results.

---

10. The Decisive Diagnosis (the only positive thing)

After all the failed approaches, we ran a focused diagnostic. The key tests:

Test 1: Sanity check at N=32

Train from scratch at N=32. BLER = 0.085 by 10K iters. **Confirms there are no implementation bugs.**

Test 2: Trained N=256 model under teacher forcing vs free running

- Teacher-forced accuracy: 99.6 percent
- Free-running BLER: 0.010
- **The trained model has essentially no rollout collapse.** The gap between teacher forcing and free running is negligible.

Test 3: Oracle injection

Take the trained model. For the first K leaf decisions, force the correct bit. Measure BLER for K = 0, 50, 100, 150, 200, 246.

K	BLER
0	0.010
50	0.000
≥50	0.000

With 50 oracle bits the BLER goes to zero. **The trained model is essentially perfect — it just has a few hard early decisions.** Errors are not cascading.

Test 4: Random model under teacher forcing at N=256

- TF accuracy: 37 percent (barely above 25 percent random baseline for 4-class output)
- TF loss: 1.36 (vs trained model 0.018)
- **Even with perfect side information, random tree ops at N=256 cannot make correct predictions.**

The conclusion

The N=256 from-scratch failure is **not** an error propagation problem. It is **not** a rollout collapse. It is **not** a bug. It is **not** a capacity issue.

It is an **optimization barrier**: at tree depth 8, random CalcLeft/CalcRight/CalcParent MLPs produce nearly uniform outputs that destroy the channel signal. The model collapses to position-constant predictions (loss=0.874, BLER=1.0) that ignore the input entirely. From this dead state there is no gradient path to a useful solution because the channel signal never reaches any leaf.

Curriculum training works because by the time you train at N=256, the tree ops are already meaningful (from N=128 training). They preserve channel information through 8 levels because they have learned to.

Confidence: 90 percent. This is supported by: 1. Overfitting test: the model can memorize 10 fixed N=256 codewords perfectly (BLER=0). So the architecture has enough capacity. 2. Random TF test: random model fails even with perfect side information. So it is not error propagation. 3. Trained model TF/FR comparison: trained model has no gap. So it is not exposure bias.

The failure mode is uniquely an optimization landscape problem at random initialization for deep trees.

---

11. Where We Stand Now

Best results

N	Best BLER	Method	Time
32	0.037	d=32 sequential	5 hours
64	0.020	d=32 sequential	14 hours
128	0.017	d=16 curriculum	~28 hours
256	0.015	d=16 curriculum from N=128	~16 hours additional
512	0.008	d=16 curriculum from N=256	~28 hours additional
1024	not done	would need ~weeks of training	—

What works

Sequential training with curriculum (N=16 → 32 → 64 → 128 → 256 → 512). This is the only training recipe that works. It is the recipe used in the d=16 results above.

What does not work

- Fast_CE for 4-class MAC (any variant)
- WHT decomposition
- Two-phase iterative refinement
- Hybrid (fast_ce pretrain → sequential fine-tune) — works at N=32, fails at N=256
- Gradient detaching
- Tree-op transfer alone (works as warm start but slower than curriculum)

What we have not finished

- d=32 model curriculum to N=256 (training was interrupted at N=128, was still improving)
- d=32 model at N=512, 1024

---

12. Possible Paths Forward

Path A: Continue d=32 curriculum to N=256+

This is the most concrete plan. Just restart the d=32 training from the N=128 checkpoint and continue to N=256, then N=512. Estimated 5-7 days of compute. Likely outcome: small improvement over d=16 baselines (0.015 → maybe 0.010 at N=256, still 2x SC).

Path B: Try even larger model (d=64)

Same architecture, more parameters. Likely gives small further improvement at the cost of much longer training.

Path C: Skip connections from root to leaves

Add direct channel-to-leaf connections so information does not have to traverse all 8 tree levels. This addresses the optimization barrier directly. Untried. Could enable from-scratch training at N=256.

Path D: Different decoder family (e.g. learned BP)

Replace the SC tree walk with belief propagation on the polar factor graph. BP is parallel by construction (O(iterations) gradient depth, not O(N log N)). Used by Nachmani et al. for LDPC. Untested for MAC polar codes. Significant engineering effort.

Path E: Accept the current state

Publish what we have. The contributions are: - First neural SC decoder for two-user MAC polar codes - Matches analytical SC at N≤128 for both BEMAC and GMAC - d=32 model beats SC at N=32, 64 - CRC-aided neural SCL achieves zero errors at N=128 - Works on channels with memory (ISI-MAC) - Detailed failure analysis at large N (the optimization barrier diagnosis is itself a contribution)

13. Quick-Reference Q&A for the Meeting

Q: Is the architecture broken? A: No. It works perfectly at N≤128 and can overfit at N=256. The architecture is sound.

Q: Why does fast_ce work for single-user but not for MAC? A: Single-user is binary (2-class). A wrong prediction flips a sign in the residual — bounded perturbation. MAC is 4-class. A wrong joint (u,v) prediction creates one of 3 distinct error patterns that the model never sees during teacher-forced training. The fast_ce assumption that "errors stay small" breaks for 4-class.

Q: Did you confirm this experimentally? A: Yes. Single-user NPD with fast_ce achieves BLER = 0.000 at N=4, 8, 16, 32. Our 4-class MAC fast_ce gives 0.45, 0.025, 0.092, 0.373 at the same N values. The 2-class case scales perfectly; the 4-class case degrades from N=16 onwards.

Q: Why does N=256 fail from scratch? A: At tree depth 8, random MLPs destroy the channel signal before it reaches the leaf. The model collapses to a degenerate fixed point (constant predictions ignoring input). There is no gradient signal to escape because no leaf receives information about the channel. We confirmed this: a random model at N=256 gets only 37 percent accuracy even with teacher forcing.

Q: Why does curriculum work? A: When you arrive at N=256 with tree ops trained at N=128, the MLPs already know how to preserve channel information through deep stacks. The optimization starts in a good basin instead of the degenerate fixed point.

Q: Is curriculum the answer? A: It is the only working answer we have, but it is not satisfying. Training time grows roughly linearly with N. Scaling to N=1024 takes weeks of compute. There is no efficient training method.

Q: Did you find anything genuinely new in this investigation? A: Three things: 1. The 4-class vs 2-class fast_ce gap is documented quantitatively across N values. 2. The optimization barrier at N=256 is diagnosed as an initialization problem, not a fundamental failure. 3. The d=32 model beats SC at small N (the only setting where the neural decoder strictly outperforms analytical SC on continuous channels).

Q: What is your recommendation? A: Either: - Finish the d=32 curriculum to N=256 (5-7 days of compute) and publish. - Or try Path C (skip connections from root to leaves) as a research idea — small POC first, full experiment if it shows promise.

14. Time Budget Summary

Approach	Compute spent	Result
Direct fast_ce	~5 days	Failed (7x SC ceiling)
WHT decomposition	~3 days	Failed (5.5x SC)
Two-phase iterative	~2 days	Failed (11x SC)
Hybrid pretrain	~3 days	Helps at N=32, fails at N=256
Gradient detaching	~1 day	Failed (no learning at all)

Approach	Compute spent	Result
d=32 curriculum	~30 hours	Beats SC at N=32, 64; not finished at N=128
Tree-op transfer	~1 hour	Works but slower than curriculum
Diagnostic experiments	~2 days	The decisive answer (optimization barrier)
Total	**~3 weeks**	**One positive contribution: clean diagnosis**

The investment was significant. The result is not what we hoped for, but the diagnosis is solid.